

dbt Training

● Working Effectively · Module 12 · 90 min

CI/CD and Slim CI

How broken models get stopped before they reach production — and why it matters how fast CI runs.

It's 3am. A Dashboard is Broken.

Slack · #data-alerts · 03:14

Pipeline failed. mart_revenue is empty. Power BI dashboard showing zeros for all of March.

GitHub · main · 2h ago

Merged: "fix deal stage mapping" — 3 files changed

What went wrong: A column rename broke a `not_null` test downstream. No CI. The code merged and ran against production.

X No CI — broken SQL reached production

X Tests existed but never ran automatically

X Someone had to find this at 3am

With CI: The PR is blocked. The merge never happens. Production never sees the broken model.

What CI/CD Means in dbt

flowchart LR

```

DEV["Developer\npushes branch"]
PR["Pull Request\nopened"]
CI["GitHub Actions\nndbt build in CI schema"]
FAIL["❌ PR blocked\nTests failed"]
REVIEW["Code review\napproved"]
MERGE["Merge to main"]
CD["dbt build\n+ docs generate\n+ upload to S3"]
PROD["✅ Production\nupdated"]

DEV --> PR --> CI
CI -- "tests pass" --> REVIEW --> MERGE --> CD --> PROD
CI -- "tests fail" --> FAIL

classDef passing fill:#ecfdf5,stroke:#16a34a,color:#065f46
classDef failing fill:#fef2f2,stroke:#dc2626,color:#7f1d1d
classDef neutral fill:#f8fafc,stroke:#94a3b8,color:#334155
class PROD,REVIEW,MERGE,CD passing

```

```
class FAIL failing
class DEV,PR,CI neutral
```

CI (Continuous Integration)

Runs on every PR push. Builds models. Runs tests. Blocks merge on failure.

CD (Continuous Delivery)

Runs on merge to main. Full build against PROD. Publishes docs to S3.

The guarantee

If CI passes, every test you wrote passed against real data. Broken models cannot merge.

PR Workflow: What Runs When

on: pull_request every PR push

▸

Download prod `manifest.json` from S3

▸

```
dbt deps
```

▸

```
dbt build --select state:modified+ --defer --state ./prod-artifacts --target ci
```

▸

Drop CI schema (cleanup)

Writes to **DBT_CI_<run-id>** — a throwaway schema. Never touches PROD.

Parallel PR safety: Unique schema per run ensures parallel PRs don't corrupt each other's test results.

on: push to main after merge

▸

```
dbt deps
```

▸

```
dbt build --target prod (full project)
```

▸

```
dbt docs generate
```

▸

Upload `manifest.json` → S3 (for next Slim CI)

▸

Sync docs site → S3

Writes to **PUBLIC** — the real production schema.

The Cost Problem: Running Everything on Every PR

Without Slim CI

```
Project: 18 models
Average build time: 3 min/model
```

```
Full dbt build: 18 × 3 = 54 min
PRs per week: 10
```

```
CI compute/week: 540 min
CI compute/month: ~2,160 min
```

Most of that compute rebuilds models that did not change.

With Slim CI (typical PR: 1 model changed)

```
Modified: fct_deal
Downstream: mart_revenue
            mart_pipeline_summary
```

```
Selected: 3 models × 3 min = 9 min
PRs/week: 10
```

```
CI compute/week: 90 min
Savings: 450 min/week (83%)
```

Same test coverage. Fraction of the cost.

As the project grows, this gap widens. 50 models × 3 min = 150 min per PR. Slim CI still builds 3–5 models per typical PR.

Slim CI: `manifest.json` + `state:modified+`

What is `manifest.json` ?

Produced by every `dbt build`. Lives in `./target/`.

Contains: the full model graph + a **compiled SQL hash** for each model.

Uploaded to S3 after each production run.

Downloaded by CI before it runs.

What does `state:modified+` do?

| Part | Selects | |`---|---`| | ``state:modified`` | Models whose SQL hash ≠ production hash | | ``+`` (trailing) | All downstream dependents | | ``--state ./prod-artifacts`` | Production manifest to compare against |

Example: you changed `fct_deal.sql`

```
flowchart TD
    STG_DEAL["STG_DEAL[\"stg_hubspot__deals\\nPROD hash = current hash\\n→ NOT selected\"]"]
    DIM_CONTACT["DIM_CONTACT[\"dim_contact\\nPROD hash = current hash\\n→ NOT selected\"]"]
    FCT_DEAL["FCT_DEAL[\"fct_deal\\nPROD hash ≠ current hash\\n→ state:modified ✓\"]"]
    MART_REV["MART_REV[\"mart_revenue\\nDownstream of fct_deal\\n→ + selector ✓\"]"]
    MART_PIPE["MART_PIPE[\"mart_pipeline_summary\\nDownstream of fct_deal\\n→ + selector ✓\"]"]

    STG_DEAL --> FCT_DEAL
    DIM_CONTACT --> FCT_DEAL
```

```

FCT_DEAL --> MART_REV
FCT_DEAL --> MART_PIPE

classDef notsel fill:#f8fafc,stroke:#cbd5e1,color:#94a3b8
classDef sel fill:#ecfdf5,stroke:#16a34a,color:#065f46
class STG_DEAL,DIM_CONTACT notsel
class FCT_DEAL,MART_REV,MART_PIPE sel

```

3 of 18 models selected. 15 skipped.

How `--defer` Works

```

flowchart TD
    STG_CONTACT["stg_hubspot__contacts\n(PROD relation)\n← deferred: CI reads this directly"]
    STG_DEAL["stg_hubspot__deals\n(PROD relation)\n← deferred: CI reads this directly"]
    DIM_CONTACT["dim_contact\n(PROD relation)\n← deferred: CI reads this directly"]
    FCT_DEAL["fct_deal\n← MODIFIED\nRebuilt in DBT_CI_4821"]
    MART_REV["mart_revenue\n← Downstream\nRebuilt in DBT_CI_4821"]
    MART_PIPE["mart_pipeline_summary\n← Downstream\nRebuilt in DBT_CI_4821"]

    STG_CONTACT --> DIM_CONTACT --> FCT_DEAL
    STG_DEAL --> FCT_DEAL
    FCT_DEAL --> MART_REV
    FCT_DEAL --> MART_PIPE

    classDef prod fill:#f0f9ff,stroke:#0369a1,color:#0c4a6e
    classDef rebuilt fill:#ecfdf5,stroke:#16a34a,color:#065f46
    class STG_CONTACT,STG_DEAL,DIM_CONTACT prod
    class FCT_DEAL,MART_REV,MART_PIPE rebuilt

```

Without `--defer`

CI must rebuild staging and dim models just to have input data. 15 extra models for a 3-model change.

With `--defer`

CI reads unmodified upstream models directly from PROD. Builds only what changed + downstream.

GitHub Actions Walkthrough

```
``yaml {all|1-5|7-14|16-24|26-36|38-44|all}
```

`.github/workflows/dbt_ci.yml`

name: dbt CI/CD

```

on:
  pull_request:
  branches: [main]
  push:
  branches: [main]

jobs:
  slim_ci:
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    env:
      DBT_TARGET: ci
      SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
      SNOWFLAKE_USER: ${ secrets.SNOWFLAKE_USER }
      SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }
      SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }
      SNOWFLAKE_WAREHOUSE: ${ secrets.SNOWFLAKE_WAREHOUSE }
      SNOWFLAKE_ROLE: ${ secrets.SNOWFLAKE_ROLE }
      DBT_SCHEMA: DBT_CI_${ github.run_id }

```

```

steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with: { python-version: '3.11' }
  - run: pip install dbt-snowflake==1.8.4
  - name: Download production manifest
    run: |
      mkdir -p ./prod-artifacts
      aws s3 cp s3://bloomwell-dbt-artifacts/prod/manifest.json \
        ./prod-artifacts/manifest.json
  - run: dbt deps
  - name: dbt build – slim CI
    run: |
      dbt build --select state:modified+ \
        --defer --state ./prod-artifacts --target ci
  - name: Drop CI schema
    if: always()
    run: dbt run-operation drop_schema \
      --args '{"schema": '$DBT_SCHEMA'}' --target ci

```

```

<!--
Use line highlights – click through each section:
1. The on: triggers – pull_request vs push to main
2. The env block – note every credential uses secrets.* – never a hardcoded value
3. DBT_SCHEMA: DBT_CI_${ github.run_id } – unique per run, enables parallel CI runs
4. Download manifest step – this is where the prod baseline comes from

```

5. The dbt build command – state:modified+, --defer, --state pointing to prod-artifacts
6. Drop CI schema with if: always() – cleanup even on failure

Ask after the secrets section: "Why does the password use `{{ secrets.SNOWFLAKE_PASSWORD }}` instead of the actual password?" → The YAML is committed to the repo. Anyone who can read the code can see the YAML. Secrets are stored in GitHub's encrypted secrets store and injected at runtime – never visible in code or logs.

Ask after the manifest step: "What would happen if this step failed – if S3 was unreachable?" → `dbt build --select state:modified+` would fail because there's no `manifest.json` to compare against. It would error with a missing state path.

-->

Environment Variables and Targets

```
<div class="grid grid-cols-2 gap-6 mt-4">
```

```
<div class="bg-white border border-slate-200 rounded-xl p-4">
```

```
<div class="text-xs font-semibold text-slate-500 mb-3">profiles.yml (committed – no secrets)</div>
```

```
```yaml
```

```
analytics:
```

```
 target: dev
```

```
 outputs:
```

```
 dev:
```

```
 type: snowflake
```

```
 account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
```

```
 user: "{{ env_var('SNOWFLAKE_USER') }}"
```

```
 password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
```

```
 schema: "TESTING__{{ env_var('DBT_DEVELOPER_NAME', 'dev') }}"
```

```
 ci:
```

```
 type: snowflake
```

```
 account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
```

```
 user: "{{ env_var('SNOWFLAKE_USER') }}"
```

```
 password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
```

```
 schema: "{{ env_var('DBT_SCHEMA', 'DBT_CI') }}"
```

```
 prod:
```

```
 type: snowflake
```

```
 account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
```

```
 user: "{{ env_var('SNOWFLAKE_USER') }}"
```

```
 password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
```

```
 schema: PUBLIC
```

Three targets, three schemas

| Target | Used by | Schema | |---|---|---| | `dev` | Developer locally | `TESTING\_\_yourname` | | `ci` | GitHub Actions (PR) |  
`DBT\_CI` | | `prod` | GitHub Actions (merge) | `PUBLIC` |

**Rule:** `profiles.yml` is committed. Never put a password in it. All credentials go through `env_var()`. Local devs set these in a `.env` file (git-ignored). CI reads from GitHub secrets.

**Key point:** CI uses target `ci`. The `--target ci` flag in the workflow tells dbt which output block to use. It never accidentally writes to `PUBLIC`.

## Documentation Quality Gate

A PR is blocked if any of these are missing:

X A Silver or Gold model has no `description` in `schema.yml`

X The SQL file has no `-- grain:` comment at the top

X A `_key` column (surrogate or FK) has no `description`

How it is enforced

The reviewer runs the `dbt-sql-reviewer` skill before approving. Every failed check becomes a review comment. The PR cannot be approved until all checks pass.

A model that passes the gate

```
-- grain: one row per deal per day
-- Each row represents the state of a HubSpot deal
-- as recorded at the end of the calendar day.

{{ config(
 materialized = 'incremental',
 unique_key = 'fct_deal_key',
 on_schema_change = 'sync_all_columns'
) }}
```

```
models:
- name: fct_deal
 description: >
 Daily snapshot of HubSpot deal state. One row per deal
 per calendar day. Source of truth for pipeline reporting.
 columns:
 - name: fct_deal_key
 description: "Surrogate key – hash of deal_id + snapshot_date"
 - name: deal_id
 description: "HubSpot deal ID"
```

layout: default

background: '#f9f8f5'

## Exercise — Fix the Broken CI Config (25 min)

**Scenario:** A colleague wrote the CI workflow below. It looks mostly right but has **three bugs**. Identify each one, explain why it is a problem, and write the fix. Then complete Tasks 2 and 3.

```
``yaml {all}
jobs:
 slim_ci:
 runs-on: ubuntu-latest
 env:
 DBT_TARGET: ci
 SNOWFLAKE_ACCOUNT: ${ secrets.SNOWFLAKE_ACCOUNT }
 SNOWFLAKE_USER: ${ secrets.SNOWFLAKE_USER }
 SNOWFLAKE_PASSWORD: "Snowflake_Prod_2024_v3" # ← ?
 SNOWFLAKE_DATABASE: ${ secrets.SNOWFLAKE_DATABASE }
 DBT_SCHEMA: DBT_CI_${ github.run_id }
 steps:
 - uses: actions/checkout@v4
 - uses: actions/setup-python@v5
 with: { python-version: '3.11' }
 - run: pip install dbt-snowflake==1.8.4
 - name: Download production manifest
 run: |
 mkdir -p ./prod-artifacts
 aws s3 cp s3://bloomwell-dbt-artifacts/prod/manifest.json ./prod-artifacts/manifest.json
 - run: dbt deps
 - name: dbt build — slim CI
 run: |
 dbt run \ # ← ?
 --select state:modified+ \
 --defer \
 --state ./target \ # ← ?
 --target ci
```

```
<div class="mt-3 grid grid-cols-2 gap-3 text-sm">
 <div class="bg-white border border-slate-200 rounded-lg p-3">
 <div class="font-semibold text-slate-700 mb-1">Task 2</div>
 <div class="text-slate-600">Write the exact <code>dbt build</code> command for a PR that only modifies
 <code>fct_deal.sql</code>. Use Slim CI with defer. State path: <code>./prod-artifacts</code>.</div>
 </div>
 <div class="bg-white border border-slate-200 rounded-lg p-3">
 <div class="font-semibold text-slate-700 mb-1">Task 3</div>
 <div class="text-slate-600">Answer in one sentence each: (1) what does <code>--defer</code> do for an
 unmodified upstream model? (2) why does this matter for CI compute cost? (3) what would happen without
 <code>--defer</code> if only one Gold mart changed?</div>
 </div>
```

```

</div>

<!--
BUGS – 3 total:

Bug 1: SNOWFLAKE_PASSWORD: "Snowflake_Prod_2024_v3"
 The workflow file is committed to the repo. The password is visible to anyone with read access.
 Fix: SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }}

Bug 2: dbt run
 dbt run builds models but does not run tests. CI exists to catch test failures. dbt run defeats the entire
 purpose.
 Fix: dbt build

Bug 3: --state ./target
 ./target is the local compiled output directory for the current run – it starts empty. Slim CI needs the
 production manifest.json, which was downloaded to ./prod-artifacts. Pointing --state at ./target means dbt
 has no baseline to compare against and either errors or treats everything as modified.
 Fix: --state ./prod-artifacts

Walk around during the exercise. Bug 2 is the easiest to spot. Bug 3 requires understanding what --state
actually reads. Bug 1 is the most serious from a security perspective – it's the one they should catch
fastest in a real code review.
-->

layout: default
background: '#f9f8f5'

Exercise – Solutions

<div class="grid grid-cols-3 gap-4 mt-4">

<div class="bg-white border border-slate-200 rounded-xl p-4">
 <div class="text-xs font-mono text-slate-400 mb-1">Bug 1 · Security</div>
 <div class="text-xs text-red-600 font-semibold mb-2">❌ Hardcoded password in YAML</div>

  ```yaml
  # Wrong
  SNOWFLAKE_PASSWORD: "Snowflake_Prod_2024_v3"

  # Fixed
  SNOWFLAKE_PASSWORD: ${ secrets.SNOWFLAKE_PASSWORD }}

```

YAML is committed to the repo. Anyone with read access sees the password. GitHub also flags this as an exposed secret.

Bug 2 · Tests not running

✗ `dbt run` instead of `dbt build`

```
# Wrong
dbt run --select state:modified+

# Fixed
dbt build --select state:modified+
```

`dbt run` builds models only. Tests are skipped. CI can pass while every `not_null` test in the project fails silently.

Bug 3 · Wrong state path

✗ `--state ./target` (local, empty)

```
# Wrong
--state ./target

# Fixed
--state ./prod-artifacts
```

`./target` is the current run's output — empty at the start. The prod manifest was downloaded to `./prod-artifacts`. Wrong path = no baseline = Slim CI breaks.

Task 2 answer: `dbt build --select state:modified+ --defer --state ./prod-artifacts --target ci`

layout: center

Module 12 Complete

CI/CD pipeline

Broken models are stopped at the PR gate — never touch production.

Slim CI

`state:modified+` + `--defer` cuts CI cost by 80%+ on typical PRs.

Documentation gate

Pre-merge is the only moment that creates leverage. Use it.

● Tier 2 Complete

You've finished Working Effectively. You can build, test, document, schedule, and protect production dbt projects.

Tier 3 — Advanced Patterns

Module 13 — Advanced Testing Patterns

Module 14 — Incremental Strategies

Module 15 — SCD2 deep dive

Module 16 — Governance & Contracts

Prep for Module 13 — Advanced Testing Patterns

Q1: Difference between `not_null` and `relationships` test?

Q2: Where do you define a reusable generic test?

Q3: What does severity: warn do vs. default error?

Q4: If a production test fails overnight, how do you investigate?